

-  MockProject: PrinterHub — Hệ thống Quản lý Máy in
 - 1. Bối cảnh dự án
 - 2. Yêu cầu chức năng
 - Module A — Quản lý dữ liệu Máy in
 - Module B — Quản lý Firmware & Bảo trì
 - Module C — Hàng đợi Ticket & Sửa chữa
 - Module D — Kiến trúc & Kỹ thuật
 - Module E — File I/O & Persistence
 - Module F — Giao diện MFC (Dialog App)
 - Module G — Unit Test
 - 3. Yêu cầu kỹ thuật bắt buộc
 - 4. Cấu trúc thư mục gợi ý
 - 5. Tiêu chí chấm điểm (Grading Rubric)
 - 5.1 Thang điểm tổng quan
 - 5.2 [A] Correctness & Completeness (30 điểm)
 - 5.3 [B] C++ Technical Depth (30 điểm)
 - 5.4 [C] Design Quality (20 điểm)
 - 5.5 [D] Safety & Robustness (10 điểm)
 - 5.6 [E] Testing & Report (10 điểm)
 - 6. Performance Bands
 - 7. Checklist tự kiểm tra trước khi nộp bài
 - Chương trình
 - Kỹ thuật C++
 - Design Patterns
 - Test & Report
 - 8. Gợi ý lộ trình 10 ngày
 -  PHASE 1 — Foundation (Ngày 1–3)
 - Ngày 1 — Project Setup & Core Data Model
 - Ngày 2 — Namespace, Templates & Functions
 - Ngày 3 — File I/O, Exception & RAII
 -  PHASE 2 — STL & Algorithms (Ngày 4–5)
 - Ngày 4 — STL Containers & Queue/Stack
 - Ngày 5 — STL Algorithms, Lambda & std::function
 -  PHASE 3 — Design Patterns (Ngày 6–7)
 - Ngày 6 — Singleton, Observer & State Pattern
 - Ngày 7 — Strategy, Command/Undo-Redo & Multithreading
 -  PHASE 4 — MFC & ViewModel (Ngày 8)

- Ngày 8 — MFC Dialogs, DDX/DDV & Doc/View
-  PHASE 5 — Testing, Optimization & Report (Ngày 9–10)
 - Ngày 9 — Unit Test & Coverage
 - Ngày 10 — Optimization, Defect Review & Final Report
-  Tổng quan lộ trình
- 9. Tài liệu tham khảo trong khóa học

MockProject: PrinterHub — Hệ thống Quản lý Máy in

Khóa học: C/C++ Foundation for MFC

Thời gian làm bài: 10 ngày (có thể điều chỉnh theo lịch trainer)

Ngôn ngữ: C++ (MFC Dialog Application)

Hình thức nộp bài: Source code + báo cáo giải thích thiết kế (.md hoặc .docx)

1. Bối cảnh dự án

Công ty **TechSupport Vietnam** cung cấp dịch vụ quản lý và bảo trì máy in cho các doanh nghiệp. Họ cần một phần mềm quản lý nội bộ để:

- Theo dõi **danh sách máy in** (model, hãng, trạng thái, bảo hành)
- Quản lý **lịch sử sửa chữa, nâng cấp firmware**
- Xử lý **hàng đợi lệnh in và ticket bảo trì**
- Tự động **phát hiện lỗi thường gặp** và cảnh báo
- Hỗ trợ giao diện **MFC Dialog** để nhân viên kỹ thuật sử dụng

Bạn được giao xây dựng hệ thống **PrinterHub** — một ứng dụng C++ hoàn chỉnh tích hợp toàn bộ kiến thức đã học.

2. Yêu cầu chức năng

Module A — Quản lý dữ liệu Máy in

ID	Chức năng
A1	Đăng ký máy in mới: model, hãng (HP/Canon/Epson/Other), trạng thái (<code>enum class PrinterStatus</code>), ngày mua, số tháng bảo hành
A2	Xem danh sách máy in, lọc theo trạng thái, hãng, còn bảo hành hay không
A3	Cập nhật trạng thái máy in (Active → InService → Retired)
A4	Xuất danh sách máy in ra file <code>.csv</code> / đọc vào từ file <code>.csv</code>
A5	Xóa máy in khỏi danh sách (soft delete — đánh dấu Retired, không xóa khỏi bộ nhớ)

Module B — Quản lý Firmware & Bảo trì

ID	Chức năng
B1	Đăng ký phiên bản firmware (tên, version, dung lượng bytes, tương thích hãng)
B2	Xếp hàng đợi (queue) lệnh nâng cấp firmware cho máy in
B3	Xử lý hàng đợi: lần lượt áp dụng từng lệnh nâng cấp (FIFO)
B4	Ghi lịch sử firmware đã cài cho mỗi máy in
B5	Undo/Redo thao tác cuối (dùng stack)

Module C — Hàng đợi Ticket & Sửa chữa

ID	Chức năng
C1	Tạo ticket bảo trì: mô tả lỗi, mức độ ưu tiên (Low/Medium/High/Critical)
C2	Sắp xếp ticket theo mức độ ưu tiên (sort với lambda)
C3	Tìm kiếm ticket theo từ khóa mô tả (find với predicate <code>std::function</code>)
C4	Đóng ticket, ghi thời gian và kỹ thuật viên xử lý
C5	Thống kê số ticket theo trạng thái

Module D — Kiến trúc & Kỹ thuật

ID	Chức năng
D1	Toàn bộ code tổ chức trong namespace <code>PrinterHub</code> với các sub-namespaces
D2	Generic Repository template <code>DeviceRepository<T></code> dùng chung cho máy in và firmware
D3	Singleton <code>FirmwareCatalog</code> — catalog toàn cục cho danh mục firmware
D4	Observer pattern : các view đăng ký nhận thông báo khi danh sách máy in thay đổi
D5	Strategy pattern : chiến lược sắp xếp máy in (theo tên, theo trạng thái, theo ngày mua)
D6	State pattern : quản lý vòng đời trạng thái máy in với transitions hợp lệ
D7	Command pattern : mỗi thao tác (thêm/sửa/xóa/nâng cấp) là một Command object để hỗ trợ Undo/Redo

Module E — File I/O & Persistence

ID	Chức năng
E1	Đọc/ghi file <code>.csv</code> cho danh sách máy in (fopen/fclose hoặc fstream)
E2	Đọc/ghi file <code>.txt</code> log lịch sử firmware
E3	Validate dữ liệu khi đọc file (bắt lỗi định dạng, thiếu trường)
E4	MFC Serialization (<code>CArchive</code>) cho Doc/View persistence

Module F — Giao diện MFC (Dialog App)

ID	Chức năng
F1	Main Dialog : hiển thị danh sách máy in (CListCtrl), nút Thêm/Sửa/Xóa
F2	Add/Edit Dialog : form nhập thông tin máy in với DDX/DDV validation
F3	Firmware Dialog : xem hàng đợi và lịch sử firmware của máy đang chọn
F4	Ticket Dialog : quản lý danh sách ticket bảo trì

ID	Chức năng
F5	Message Map xử lý sự kiện: <code>ON_BN_CLICKED</code> , <code>ON_LBN_SELCHANGE</code> , <code>ON_EN_CHANGE</code>

Module G — Unit Test

ID	Chức năng
G1	Unit test cho logic đăng ký máy in (valid/invalid input)
G2	Fixture test cho vòng đời trạng thái máy in
G3	Regression test cho tính năng Undo/Redo
G4	ViewModel test: tách logic validation ra khỏi UI, test độc lập
G5	Coverage report: giải thích line coverage vs branch coverage cho ít nhất 2 module

3. Yêu cầu kỹ thuật bắt buộc

Mỗi mục dưới đây ánh xạ trực tiếp đến một chương trong khóa học. **Tất cả phải có mặt trong project.**

Chương	Yêu cầu kỹ thuật bắt buộc
Ch.1 — Variables & Data Types	Dùng <code>enum class PrinterStatus</code> , <code>enum class Priority</code> , <code>enum class Brand</code> . Giải thích tại sao dùng <code>enum class</code> thay vì <code>int</code> magic values
Ch.2 — Memory & Preprocessor	File <code>.h/.cpp</code> tách biệt rõ ràng. Dùng <code>#pragma once</code> hoặc include guard. Tách <code>PrinterConfig.h</code> (constants toàn cục). Giải thích compile/link flow
Ch.3 — Arrays, Pointers & Reference	Dùng cả raw pointer và reference trong ít nhất 1 module. Giải thích ownership và vòng đời. Không có dangling pointer
Ch.4 — Functions & Passing Variables	Có function pass-by-value, pass-by-reference, pass-by-pointer. Giải thích khi nào dùng loại nào

Chương	Yêu cầu kỹ thuật bắt buộc
Ch.5 — File I/O & Debugging	Đọc/ghi file CSV, validate input. Dùng debugger để trace ít nhất 1 bug. Ghi chú bug đã fix trong báo cáo
Ch.6 — OOP Basics	Class <code>Printer</code> có constructor/destructor rõ ràng. Implement Rule of 3 hoặc Rule of 5. Operator <code>==</code> và <code><</code> cho việc compare và sort
Ch.7 — OOP Features	Interface <code>IPrinterObserver</code> thuần ảo. <code>PrinterManager</code> kế thừa thiết kế. Dùng <code>dynamic_cast</code> đúng cách với null check
Ch.8 — Namespace & Templates	Namespace <code>PrinterHub::Core</code> , <code>PrinterHub::UI</code> , <code>PrinterHub::Tests</code> . Template class <code>DeviceRepository<T></code>
Ch.9 — Data Structures & Algorithms	<code>std::vector</code> cho danh sách chính. <code>std::map<string, Printer></code> cho lookup. <code>std::queue</code> cho hàng đợi firmware. <code>std::stack</code> cho Undo/Redo. <code>std::set</code> cho danh sách brand unique
Ch.9 (Lambda)	Sort ticket với lambda: <code>std::sort(..., [](const Ticket& a, const Ticket& b){ ... })</code> . Filter với <code>std::function<bool(const Printer&)></code>
Ch.10 — Exception Handling & RAII	Custom exception class <code>PrinterHubException</code> . RAII wrapper cho file handle. <code>unique_ptr</code> / <code>shared_ptr</code> dùng đúng chỗ
Ch.11 — Move Semantics	Implement move constructor cho class chứa dữ liệu lớn (<code>FirmwarePackage</code>). Dùng <code>std::move</code> khi insert vào container. <code>reserve()</code> cho vector lớn
Ch.12 — Unit Test	GoogleTest: ít nhất 10 test case. Có fixture (<code>TEST_F</code>). Có regression test cho bug đã fix
Ch.13 — Coverage & Testability	ViewModel class <code>PrinterFormVM</code> tách khỏi UI. Giải thích line vs branch coverage. Coverage report cho <code>PrinterFormVM</code>
Ch.14 — Multithreading	Thread worker load danh sách máy in từ file (không block UI). <code>std::mutex</code> bảo vệ shared data. <code>PostMessage/SendMessage</code> cập nhật UI từ thread

Chương	Yêu cầu kỹ thuật bắt buộc
Ch.12 Opt — Common Defects	Trong báo cáo: liệt kê ít nhất 3 defect phổ biến đã gặp/phòng tránh (dangling ref, double-free, iterator invalidation, race condition)
Ch.15 — Design Patterns (Singleton + Observer)	<code>FirmwareCatalog</code> là Singleton thread-safe. Observer: <code>PrinterManager</code> (Subject) + các Dialog (Observer)
Ch.16 — Strategy, State, Command	Strategy sort. State machine cho printer lifecycle. Command objects cho Undo/Redo
Ch.17 — MFC Dialog & DDX/DDV	Add/Edit Printer Dialog với DDX binding + DDV validation (model name không rỗng, bảo hành > 0)
Ch.18 — MFC Doc/View	<code>CPrinterDoc</code> lưu dữ liệu, <code>CPrinterView</code> hiển thị, <code>UpdateAllViews()</code> khi data thay đổi
Ch.20 — CString & Unicode	Dùng <code>CString</code> , <code>_T()</code> macro, <code>TCHAR</code> . Hàm chuyển đổi giữa <code>std::string</code> và <code>CString</code>

4. Cấu trúc thư mục gợi ý

```

PrinterHub/
├── src/
│   ├── core/                                # Business logic (no MFC dependency)
│   │   ├── Printer.h/.cpp                    # Class Printer, enum class PrinterStatus
│   │   ├── FirmwarePackage.h/.cpp
│   │   ├── MaintenanceTicket.h/.cpp
│   │   ├── PrinterManager.h/.cpp # Subject (Observer)
│   │   ├── FirmwareCatalog.h/.cpp # Singleton
│   │   ├── DeviceRepository.h               # Template repository
│   │   ├── Commands.h/.cpp                  # Command pattern (Undo/Redo)
│   │   ├── States.h/.cpp                    # State pattern (printer lifecycle)
│   │   ├── Strategies.h/.cpp                # Sort strategies
│   │   └── PrinterConfig.h                  # Constants, enums, preprocessor macros
│   ├── io/
│   │   ├── CsvReader.h/.cpp                 # File I/O
│   │   ├── CsvWriter.h/.cpp
│   │   └── FirmwareLog.h/.cpp
│   └── ui/                                  # MFC layer
│       ├── MainDlg.h/.cpp                   # Main dialog
│       ├── AddEditPrinterDlg.h/.cpp
│       ├── FirmwareDlg.h/.cpp
│       ├── TicketDlg.h/.cpp
│       └── PrinterFormVM.h/.cpp # ViewModel (testable)

```

```
| | | | PrinterDoc.h/.cpp      # MFC Doc/View
| | | | worker/
| | | | | LoadWorker.h/.cpp    # Background thread
| | tests/
| | | | PrinterTests.cpp
| | | | TicketTests.cpp
| | | | UndoRedoTests.cpp
| | | | ViewModelTests.cpp
| | data/
| | | | printers.csv
| | | | firmware_log.txt
| | PrinterHub.sln
| | report/
| | | | design_report.md      # Báo cáo thiết kế
```

5. Tiêu chí chấm điểm (Grading Rubric)

5.1 Thang điểm tổng quan

Hạng mục	Điểm tối đa
[A] Correctness & Completeness — Code chạy đúng, đủ chức năng	30
[B] C++ Technical Depth — Áp dụng đúng kỹ thuật C++ từng chương	30
[C] Design Quality — Kiến trúc, Design Patterns, OOP principles	20
[D] Safety & Robustness — Memory safety, exception, thread safety	10
[E] Testing & Report — Unit test, coverage, báo cáo thiết kế	10
TỔNG	100

5.2 [A] Correctness & Completeness (30 điểm)

Điểm	Mô tả
27–30	Tất cả module A–G hoạt động đúng. File I/O đọc/ghi chính xác. MFC dialog mở, validate, lưu dữ liệu không lỗi

Điểm	Mô tả
21–26	Hầu hết module hoạt động. 1–2 tính năng nhỏ thiếu hoặc có lỗi nhỏ không ảnh hưởng luồng chính
13–20	50–70% chức năng hoạt động. Thiếu MFC hoặc thiếu File I/O
0–12	Dưới 50% chức năng. Project không chạy được hoặc chỉ có skeleton

Chi tiết module:

Module	Điểm	Tiêu chí
A — Quản lý Máy in	8	CRUD đầy đủ, filter, soft-delete hoạt động
B — Firmware & Bảo trì	6	Queue xử lý FIFO đúng, lịch sử ghi chính xác
C — Ticket & Sửa chữa	6	Sort, search bằng predicate hoạt động đúng
E — File I/O	5	Đọc/ghi CSV không lỗi, validate bắt được định dạng sai
F — MFC Dialog	5	Ít nhất Main Dialog + Add Dialog hoạt động với DDX/DDV

5.3 [B] C++ Technical Depth (30 điểm)

Mỗi nhóm kỹ thuật được chấm độc lập. Thiếu một nhóm hoàn toàn = 0 điểm nhóm đó.

Nhóm	Điểm	Kiểm tra
B1 Variables, Enums, Scope	2	Dùng <code>enum</code> <code>class</code> , không dùng magic number int, giải thích scope
B2 Preprocessor & Compile	2	Header guard/pragma once, tách .h/.cpp, explain compile flow trong report
B3 Pointers & Reference	3	Có raw pointer + reference. Không dangling pointer. Null check rõ ràng

Nhóm	Điểm	Kiểm tra
B4 Functions & Passing	2	Đủ 3 kiểu: by value, by ref, by pointer. Có const correctness
B5 File I/O & Debugging	2	fopen/fstream dùng đúng. Validate đọc file. Ghi chú ít nhất 1 debug session
B6 OOP Basics (Rule of 3/5)	3	Copy constructor + operator= + destructor. Operator < và ==
B7 OOP Features	3	Interface thuần ảo, virtual destructor, dynamic_cast với null check
B8 Namespace & Templates	3	Namespace tổ chức rõ. Template <code>DeviceRepository<T></code> generic hoạt động
B9 STL & Lambda	4	vector/map/queue/stack/set dùng đúng container. Lambda sort. <code>std::function</code> filter
B10 Exception & RAII	3	Custom exception. RAII file/resource. smart pointer đúng chỗ
B11 Move Semantics	2	Move constructor/assignment. <code>std::move</code> . <code>reserve()</code>
B12 Multithreading	1	Worker thread + mutex + PostMessage cập nhật UI

5.4 [C] Design Quality (20 điểm)

Tiêu chí	Điểm	Mô tả
C1 Singleton Pattern	3	<code>FirmwareCatalog</code> — thread-safe, lazy init, giải thích đúng
C2 Observer Pattern	4	Subject/Observer tách biệt. Unsubscribe không crash. Realistic notification flow
C3 Strategy Pattern	3	Ít nhất 2 strategy sort. Dễ thêm strategy mới (Open/Closed)
C4 State Pattern	4	State machine rõ ràng. Transition hợp lệ được enforce. Transition không hợp lệ bị từ chối

Tiêu chí	Điểm	Mô tả
C5 Command & Undo/Redo	3	Command object. Execute + Undo. Stack-based history. Redo stack xóa khi có command mới
C6 ViewModel Testability	3	PrinterFormVM tách khỏi MFC. Logic validation test được không cần UI

5.5 [D] Safety & Robustness (10 điểm)

Tiêu chí	Điểm	Mô tả
D1 Memory Safety	4	Không leak (dùng Valgrind, AddressSanitizer, hoặc smart pointer). Không double-free
D2 Exception Safety	3	Không crash khi input sai. File không tồn tại xử lý graceful. Exception không rò resource
D3 Thread Safety	2	Mutex bảo vệ shared data. Không update UI trực tiếp từ worker thread
D4 Defect Report	1	Báo cáo liệt kê ít nhất 3 loại defect đã phòng tránh với cách fix

5.6 [E] Testing & Report (10 điểm)

Tiêu chí	Điểm	Mô tả
E1 Unit Tests (Google Test)	4	Ít nhất 10 test case. Có TEST_F fixture. Test cover valid + invalid input
E2 Regression Test	2	Ít nhất 1 regression test cho bug đã fix. Test name mô tả scenario rõ ràng
E3 Coverage Analysis	2	Giải thích line vs branch coverage. Chỉ ra uncovered branch và lý do
E4 Design Report	2	Report trình bày kiến trúc, quyết định thiết kế, trade-off đã chọn

6. Performance Bands

Band	Điểm	Mô tả
 Excellent	90–100	Code chạy hoàn chỉnh. Thiết kế tốt, giải thích rõ lý do chọn design. Safety và testability tốt. Báo cáo chuyên nghiệp
 Good	75–89	Phần lớn chức năng đúng. Có vài điểm design chưa tối ưu. Test đủ nhưng coverage chưa sâu
 Needs Improvement	55–74	50–70% chức năng. Áp dụng kỹ thuật chưa đầy đủ. Thiếu test hoặc thiếu báo cáo
 Unsatisfactory	< 55	Dưới 50% chức năng hoặc không chạy được. Kỹ thuật C++ áp dụng sai. Thiếu nhiều module

7. Checklist tự kiểm tra trước khi nộp bài

Sinh viên tự check trước khi submit:

Chương trình

- ☐ Project build thành công, không có warning nghiêm trọng
- ☐ Chương trình chạy, thêm/sửa/xóa/tìm máy in hoạt động
- ☐ Đọc/ghi file CSV hoạt động đúng
- ☐ MFC Dialog hiển thị danh sách, Add Dialog validate input
- ☐ Hàng đợi firmware xử lý FIFO
- ☐ Undo/Redo hoạt động ít nhất 3 thao tác

Kỹ thuật C++

- ☐ `enum class` dùng thay thế magic int
- ☐ Header/source tách biệt, có include guard
- ☐ Không có dangling pointer hay memory leak hiển nhiên

- ☐ Rule of 3 hoặc 5 implement cho class có resource
- ☐ Interface thuần ảo với virtual destructor
- ☐ Template `DeviceRepository<T>` generic
- ☐ STL: vector, map, queue, stack, set đều có mặt
- ☐ Lambda trong sort/filter
- ☐ Custom exception và RAII
- ☐ Smart pointer (`unique_ptr/shared_ptr`) dùng đúng chỗ
- ☐ Move semantics cho class lớn

Design Patterns

- ☐ Singleton `FirmwareCatalog` hoạt động đúng
- ☐ Observer: thêm máy in → Dialog cập nhật tự động
- ☐ Strategy: có thể đổi cách sort không cần sửa `PrinterManager`
- ☐ State machine: transition không hợp lệ bị từ chối
- ☐ Command + Undo/Redo: stack history đúng

Test & Report

- ☐ Ít nhất 10 test case GoogleTest pass
- ☐ Có `TEST_F` fixture
- ☐ Có 1 regression test
- ☐ Báo cáo thiết kế: giải thích ít nhất 3 quyết định kỹ thuật
- ☐ Báo cáo: liệt kê defect đã phòng tránh

8. Gợi ý lộ trình 10 ngày

Mỗi ngày tương đương **~4 giờ thực hành**. Cuối mỗi phase có **checkpoint** tự kiểm tra trước khi sang phase tiếp theo.

PHASE 1 — Foundation (Ngày 1–3)

Mục tiêu: Dựng khung dự án, hoàn thiện toàn bộ core business logic.

Ngày 1 — Project Setup & Core Data Model

#	Nhiệm vụ	Chương
1	Tạo solution <code>PrinterHub.sln</code> . Cấu trúc thư mục <code>src/core</code> , <code>src/io</code> , <code>src/ui</code> , <code>tests/</code>	Ch.2
2	Tạo <code>PrinterConfig.h</code> : định nghĩa <code>enum class PrinterStatus</code> , <code>enum class Brand</code> , <code>enum class Priority</code>	Ch.1
3	Viết <code>Printer.h/.cpp</code> : fields, constructor có default value, destructor, copy constructor (Rule of 3)	Ch.6
4	Implement <code>operator==</code> , <code>operator<</code> , <code>operator<<</code> cho <code>Printer</code>	Ch.6
5	Viết <code>FirmwarePackage.h/.cpp</code> : move constructor + move assignment (Rule of 5)	Ch.11
6	Viết <code>MaintenanceTicket.h/.cpp</code> : fields, constructor, <code>enum class Priority</code>	Ch.1, Ch.6

✔ **Checkpoint Ngày 1:** Tất cả class compile không lỗi. `Printer p1 = p2` (copy) hoạt động đúng. `FirmwarePackage f = std::move(f2)` không crash.

Ngày 2 — Namespace, Templates & Functions

#	Nhiệm vụ	Chương
1	Bọc toàn bộ core class trong namespace <code>PrinterHub::Core</code>	Ch.8
2	Tạo <code>DeviceRepository<T></code> : Add, Remove, FindById, GetAll (template header-only)	Ch.8
3	Viết ít nhất 3 hàm pass-by-value, pass-by-reference, pass-by-pointer trong <code>PrinterUtils.h</code>	Ch.4
4	Viết <code>PrinterManager.h/.cpp</code> : dùng <code>DeviceRepository<Printer></code> làm storage chính	Ch.8
5	Viết <code>main_test.cpp</code> (unit test file): thêm 3 máy in vào <code>DeviceRepository</code> , <code>assert count == 3</code> , xóa 1 máy, <code>assert count == 2</code>	Ch.4

✔ **Checkpoint Ngày 2:** `DeviceRepository<Printer>` và `DeviceRepository<FirmwarePackage>` hoạt động song song. Namespace không bị

conflict. (Không cần console — verify qua assert hoặc GoogleTest sơ bộ)

Ngày 3 — File I/O, Exception & RAII

#	Nhiệm vụ	Chương
1	Tạo <code>PrinterHubException</code> kế thừa <code>std::exception</code> . Định nghĩa <code>FileNotFoundException</code> , <code>InvalidDataException</code>	Ch.10
2	Viết <code>CsvReader.h/.cpp</code> : đọc <code>printers.csv</code> , validate từng dòng (bắt exception nếu thiếu field)	Ch.5, Ch.10
3	Viết <code>CsvWriter.h/.cpp</code> : ghi danh sách máy in ra file	Ch.5
4	Tạo RAII wrapper <code>FileGuard</code> cho <code>FILE*</code> (constructor mở file, destructor đóng file)	Ch.10
5	Viết <code>FirmwareLog.h/.cpp</code> : append log lịch sử firmware vào file <code>.txt</code>	Ch.5
6	Dùng debugger trace 1 lỗi parsing. Ghi note vào <code>report/debug_log.md</code>	Ch.5
7	Implement <code>unique_ptr<Printer></code> ở nơi phù hợp, <code>shared_ptr</code> cho <code>FirmwareCatalog</code>	Ch.10

✔ **Checkpoint Phase 1:** Đọc file CSV → load vào `DeviceRepository` → ghi lại ra CSV khác, dữ liệu không mất. Exception bắt được file không tồn tại.

 PHASE 2 — STL & Algorithms (Ngày 4–5)

Mục tiêu: Trang bị đầy đủ container và thuật toán STL cho các nghiệp vụ chính.

Ngày 4 — STL Containers & Queue/Stack

#	Nhiệm vụ	Chương
1	<code>PrinterManager</code> dùng <code>std::vector<Printer></code> là danh sách chính	Ch.9
2	Thêm <code>std::map<std::string, Printer></code> để lookup nhanh theo ID	Ch.9
3	Thêm <code>std::set<Brand></code> để lưu danh sách brand đang có	Ch.9

#	Nhiệm vụ	Chương
4	<code>FirmwareQueue</code> : dùng <code>std::queue<FirmwarePackage></code> xử lý FIFO — enqueue, dequeue, peek	Ch.9
5	<code>UndoRedoManager</code> : dùng <code>std::stack<ICommand*></code> (undo stack) + <code>std::stack<ICommand*></code> (redo stack) — skeleton	Ch.9
6	Gọi <code>vector.reserve()</code> trước khi load batch máy in từ file	Ch.11

✅ **Checkpoint Ngày 4:** Queue firmware xử lý đúng FIFO. Map lookup O(1). Set không có brand trùng.

Ngày 5 — STL Algorithms, Lambda & `std::function`

#	Nhiệm vụ	Chương
1	Sort danh sách máy in theo tên bằng lambda: <code>std::sort(vec.begin(), vec.end(), [](...){})</code>	Ch.9
2	Sort ticket theo Priority với lambda	Ch.9
3	Filter máy in theo <code>std::function<bool(const Printer*)></code> predicate	Ch.9
4	Dùng <code>std::find_if</code> tìm ticket theo từ khóa mô tả	Ch.9
5	Viết <code>PrinterFilter</code> utility: nhận <code>std::function</code> làm tham số, trả <code>std::vector<Printer></code> kết quả	Ch.9
6	Viết <code>std::move</code> khi push <code>FirmwarePackage</code> vào queue, tránh copy không cần thiết	Ch.11

✅ **Checkpoint Phase 2:** Có thể lọc "tất cả máy in HP còn bảo hành" bằng 1 predicate. Ticket list sort đúng thứ tự Critical → High → Medium → Low.

PHASE 3 — Design Patterns (Ngày 6–7)

Mục tiêu: Áp dụng 4 Design Pattern + Observer, Multithreading.

Ngày 6 — Singleton, Observer & State Pattern

#	Nhiệm vụ	Chương
1	<code>FirmwareCatalog</code> Singleton: lazy init, thread-safe với <code>std::call_once</code>	Ch.15
2	Interface <code>IPrinterObserver</code> thuần ảo: <code>virtual void OnPrinterListChanged() = 0</code>	Ch.7, Ch.15
3	<code>PrinterManager</code> (Subject): <code>Subscribe</code> , <code>Unsubscribe</code> , <code>Notify</code>	Ch.15
4	State classes: <code>ActiveState</code> , <code>InServiceState</code> , <code>RetiredState</code> kế thừa <code>IPrinterState</code>	Ch.16
5	<code>Printer</code> giữ con trỏ <code>IPrinterState*</code> — transition hợp lệ: <code>Active→InService→Retired</code> . Transition ngược bị từ chối (throw exception)	Ch.16
6	Dùng <code>dynamic_cast</code> để detect loại state hiện tại, kèm null check	Ch.7

✅ **Checkpoint Ngày 6:** Gọi `printer.Transition(RetiredState)` từ `ActiveState` trực tiếp → exception. Gọi đúng thứ tự → thành công.

Ngày 7 — Strategy, Command/Undo-Redo & Multithreading

#	Nhiệm vụ	Chương
1	Interface <code>ISortStrategy</code> : <code>virtual std::vector<Printer> Sort(...)</code>	Ch.16
2	Implement: <code>SortByName</code> , <code>SortByStatus</code> , <code>SortByPurchaseDate</code>	Ch.16
3	<code>PrinterManager::SetSortStrategy()</code> — đổi strategy runtime không cần sửa code	Ch.16
4	Interface <code>ICommand</code> : <code>Execute()</code> , <code>Undo()</code> . Implement <code>AddPrinterCommand</code> , <code>DeletePrinterCommand</code> , <code>UpdateFirmwareCommand</code>	Ch.16
5	<code>UndoRedoManager</code> : <code>Execute</code> → push undo stack, clear redo. <code>Undo</code> → pop undo, push redo. <code>Redo</code> → pop redo, push undo	Ch.16
6	<code>LoadWorker</code> : <code>std::thread</code> load CSV trong background, dùng <code>std::mutex</code> bảo vệ shared list	Ch.14

#	Nhiệm vụ	Chương
7	Sau khi load xong: dùng <code>PostMessage</code> hoặc callback để thông báo UI (không update UI trực tiếp từ thread)	Ch.14

✅ **Checkpoint Phase 3:** Undo/Redo 3 lần liên tiếp đúng. Đổi strategy sort không cần restart. Singleton chỉ tạo 1 instance.

PHASE 4 — MFC & ViewModel (Ngày 8)

Mục tiêu: Xây dựng giao diện MFC kết nối với toàn bộ core logic.

Ngày 8 — MFC Dialogs, DDX/DDV & Doc/View

#	Nhiệm vụ	Chương
1	Tạo MFC Dialog Application project (không phải console). Link với <code>src/core</code> và <code>src/io</code>	Ch.17
2	<code>MainDlg</code> (dialog chính, khởi động ngay khi mở app): <code>CListCtrl</code> hiển thị danh sách máy in. Toolbar/nút Thêm, Sửa, Xóa, Undo, Redo	Ch.17
3	<code>MainDlg</code> implement <code>IPrinterObserver::OnPrinterListChanged()</code> — tự refresh khi dữ liệu thay đổi	Ch.15, Ch.17
4	<code>AddEditPrinterDlg</code> : DDX binding cho Model Name, Brand (CComboBox), Warranty Months	Ch.17
5	DDV validation: model name không rỗng, warranty > 0, ngày mua hợp lệ	Ch.17
6	Message Map: <code>ON_BN_CLICKED(IDC_BTN_ADD)</code> , <code>ON_LBN_SELCHANGE</code> , <code>ON_EN_CHANGE</code>	Ch.17
7	<code>PrinterFormVM</code> : class tách khỏi MFC, chứa validate logic — gọi từ cả dialog và unit test	Ch.13, Ch.17
8	<code>CPrinterDoc</code> / <code>CPrinterView</code> : <code>Serialize()</code> với <code>CArchive</code> , <code>UpdateAllViews()</code> khi data thay đổi	Ch.18

#	Nhiệm vụ	Chương
9	Dùng <code>CString</code> , <code>_T()</code> , <code>TCHAR</code> . Viết hàm <code>ToStdString(CString)</code> và <code>ToCString(std::string)</code>	Ch.17, Ch.20

✓ **Checkpoint Phase 4:** Mở app → danh sách load. Thêm máy in → list tự cập nhật. Add dialog validate reject model name rỗng. Undo xóa máy vừa thêm.

PHASE 5 — Testing, Optimization & Report (Ngày 9–10)

Mục tiêu: Đảm bảo chất lượng, tối ưu và tổng kết.

Ngày 9 — Unit Test & Coverage

#	Nhiệm vụ	Chương
1	Setup GoogleTest project, link với <code>src/core</code>	Ch.12
2	<code>PrinterTests.cpp</code> : test <code>AddPrinter</code> (valid), test <code>AddPrinter</code> (invalid model name)	Ch.12
3	<code>PrinterTests.cpp</code> : test copy constructor (deep copy), test <code>operator==</code>	Ch.12
4	<code>TEST_F(PrinterLifecycleTest, ...)</code> : fixture setup <code>Printer</code> ở Active. Test transition valid + invalid	Ch.12
5	<code>UndoRedoTests.cpp</code> : test <code>Execute→Undo→state restored</code> . Test <code>Execute→Undo→Redo→state restored again</code>	Ch.12
6	<code>ViewModelTests.cpp</code> : test <code>PrinterFormVM::Validate()</code> với model rỗng → false. Với model hợp lệ → true	Ch.13
	Regression test: đặt tên	
7	<code>TEST(Regression_Issue42_UndoAfterRetiredShouldNotCrash, ...)</code>	Ch.12
8	Viết coverage analysis trong report: chỉ ra uncovered branch trong <code>CsvReader</code>	Ch.13

✅ **Checkpoint Ngày 9:** 10+ test case đều PASS. Có ít nhất 1 **TEST_F** và 1 regression test.

Ngày 10 — Optimization, Defect Review & Final Report

#	Nhiệm vụ	Chương
1	Review toàn bộ code: kiểm tra dangling pointer, double-free, iterator invalidation	Ch.12 Opt
2	Kiểm tra thread safety: không có race condition trong LoadWorker + PrinterManager	Ch.14
3	Profiling đơn giản: đo thời gian load 1000 máy in từ file, so sánh với/không có reserve()	Ch.11
4	Viết report/design_report.md : mô tả ít nhất 3 quyết định thiết kế + lý do + trade-off	Ch.12
5	Viết report/defect_log.md : liệt kê ≥ 3 defect đã phòng tránh (dangling ref, double-free, race condition, iterator invalidation) + cách fix	Ch.12 Opt
6	Self-check toàn bộ checklist mục 7	Tất cả
7	Build Release, chạy lần cuối, kiểm tra crash corner case (file rỗng, input sai định dạng, Undo khi stack rỗng)	Tất cả
8	Đóng gói nộp bài: source + data + tests + report	Tất cả

✅ **Checkpoint Phase 5 (Final):** Build Release không warning. 10+ test PASS. Report hoàn chỉnh. Self-check ≥ 90% mục đã tick.

 Tổng quan lộ trình

Phase	Ngày	Nội dung chính	Deliverable
 Foundation	1–3	Core classes, Namespace, Template, File I/O, Exception, RAII	src/core/ + src/io/ hoàn chỉnh

Phase	Ngày	Nội dung chính	Deliverable
 STL & Algo	4–5	Vector, Map, Queue, Stack, Set, Lambda, std::function	DeviceRepository + PrinterFilter chạy được
 Patterns	6–7	Singleton, Observer, State, Strategy, Command, Multithreading	Undo/Redo + Observer notification hoạt động
 MFC	8	Dialog, DDX/DDV, Message Map, Doc/View, CString	App chạy được với UI đầy đủ
 QA & Report	9–10	Unit Test, Coverage, Optimization, Defect Review, Report	10+ tests PASS + report hoàn chỉnh

9. Tài liệu tham khảo trong khóa học

Module cần xem lại	Tài liệu
Variables, Enum, Scope	Variables & Data Types/material.md
Preprocessor, Compile	Memory Layout & Preprocessor & Compile/material.md
Pointer, Reference	Arrays, Pointers & Reference/memory_pointer_reference/material.md
File I/O, Debugging	File I O (FILE) & Debugging/material.md
OOP Basics	OOP Basics/class_constructor_destructor/material.md
OOP Features	OOP Feature/inheritance_polymorphism/material.md
Namespace, Templates	Namespace & Templates/templates_generic_repository/material.md
STL, Lambda	Data Structures & Algorithms (C & C++)/stl_algorithms_sort_find/material.md

Module cần xem lại	Tài liệu
Exception, RAI	Advanced C++ Features/raii_smart_pointers/material.md
Move Semantics	Optimization & Common Defects/move_semantics_optimization/material.md
Unit Test	Unit Test/unit_test_googletest/material.md
Design Patterns	Design Patterns/singleton_pattern/material.md
MFC Dialog	Design Patterns/mfc_dialog_message_map/material.md
MFC Doc/View	Design Patterns/mfc_doc_view/material.md

Lưu ý cho Trainer: MockProject này được thiết kế để đánh giá toàn bộ 30 topics trong 13 chương. Có thể điều chỉnh trọng số điểm theo mức độ học phần ưu tiên. Phần MFC (Module F) có thể giảm điểm nếu khóa học không đủ thời gian đến MFC, chuyển trọng số sang [B] Technical Depth.